

Documento de reflexión — Taller básico de microservicios

Electiva I — Arquitectura de Microservicios con Spring Boot · Ecosistema mínimo: `producto-service` (8081) y `pedido-service` (8082)

¿Qué responsabilidad tiene cada microservicio?

producto-service es dueño del catálogo. Gestiona la creación, consulta y listado de productos, junto con sus datos de precio y stock. Es la única fuente de verdad sobre los productos y su base de datos (`productodb`) solo la toca él.

pedido-service es dueño de los pedidos. Registra qué producto se pidió, en qué cantidad, el total y el estado. No conoce los datos internos de un producto: cuando necesita el precio para calcular el total, lo pide a `producto-service` por HTTP. Su base de datos (`pedidodb`) es independiente.

Cada servicio tiene una sola razón para cambiar (responsabilidad única) y su propia base de datos, lo que mantiene el acoplamiento bajo.

¿Qué pasaría si necesitáramos escalar solo `producto-service`?

En un monolito habría que desplegar más copias de toda la aplicación aunque el cuello de botella fuera solo el catálogo. Aquí no: `producto-service` es un ejecutable independiente, así que se pueden levantar varias instancias suyas (por ejemplo detrás de un balanceador) sin tocar ni redeplegar `pedido-service`. Se paga cómputo solo donde hace falta. El límite actual es que `pedido-service` apunta a una URL fija (`http://localhost:8081`), por lo que para repartir carga entre varias instancias haría falta un balanceador o un mecanismo de descubrimiento de servicios.

¿Qué limitación notaron al tener la URL del otro servicio escrita directamente en `application.yml`?

La dirección `http://localhost:8081` queda fija en el código de configuración de `pedido-service`. Eso implica:

- Si `producto-service` cambia de host o de puerto, hay que editar el `application.yml` y volver a desplegar `pedido-service`.
- No sirve para varios ambientes (local, pruebas, producción) sin mantener archivos distintos.
- No permite tener varias instancias de `producto-service` y repartir las peticiones entre ellas.
- `pedido-service` necesita saber de antemano dónde está exactamente el otro servicio.

Esta rigidez es la que resuelven Config Server (externalizar la URL, Semana 6) y luego Eureka / Service Discovery (que `pedido-service` encuentre a `producto-service` por su nombre, sin conocer host ni puerto, Semana 7).